Data Analysis Project
Final Handin
(Group 05)

Simon Zeugin, Max Reinert, and Alexander Marggraf

University of Basel
Databases (CS246) course
Autumn Semester 2025

1 Introduction

The goal of our project was to identify interesting datasets, create a consistent schema that includes all data sources, clean and integrate the data into a single database, and analyze the data to gain meaningful insights. Throughout this process, we aimed not only to uncover interesting facts about our chosen datasets and the real-world circumstances they model, but also to develop knowledge and skills in finding, cleaning, and integrating real-world, messy data into one coherent database.

The datasets we selected consisted of train and weather data from Switzerland and Germany, which we obtained from the internet. [7][5][4][2] The German train data was provided by Deutsche Bahn and consists of a collection of `.parquet` files which list all recorded train stops of trains operated by Deutsche Bahn in Germany. The collection was already processed such that there is one file per month, which records, among other things, where and when each train stopped and whether it was delayed or canceled. An ER diagram can be seen in Figure 1.

The German weather data was provided by the Deutsche Wetterdienst and consists of a complex collection of text files which are structured similarly to CSV files. These files are separated by measurement type, including temperature, precipitation, pressure, and wind, as well as by historical data up to 2025 and recent data. The files themselves contain the corresponding measurements, the time at which each measurement was taken, and the station at which it was recorded. Additionally, the Deutsche Wetterdienst provided metadata for the weather stations, which offers insight into their individual locations. An ER diagram can be seen in Figure 2.

The Swiss train data was provided by SBB and consists of a collection of large CSV files which list all recorded train stops of trains operated by SBB on a monthly basis. This includes, among other information, the locations at which trains stopped, their exact arrival and departure times, and whether the trains were canceled. An ER diagram can be seen in Figure 3.

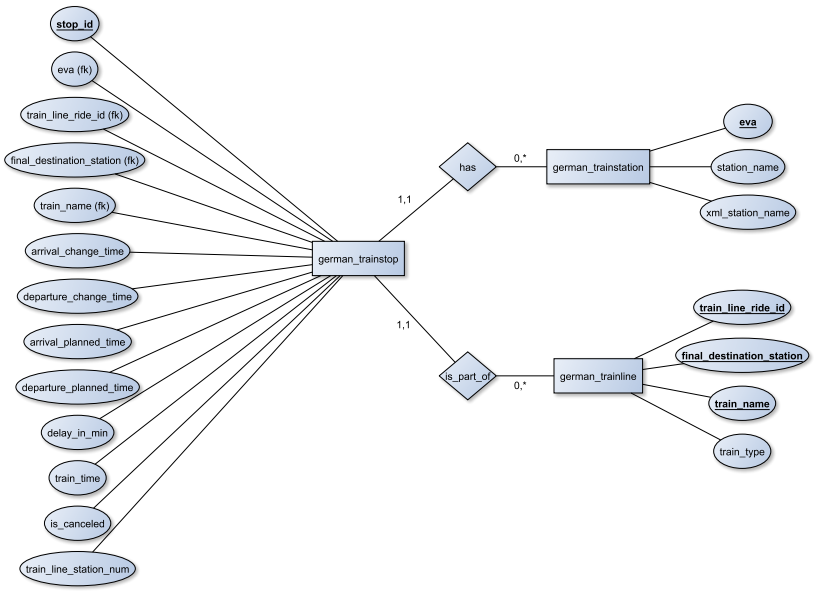


Fig. 1: ER Diagram of the german train dataset

The Swiss weather data was provided by MeteoSchweiz and consists of a collection of large CSV files which list all measured meteorological variables, such as temperature or precipitation, when these measurements were taken, and which station recorded them. Additionally, we obtained a list of all Swiss weather stations recorded by MeteoSchweiz, which can be found here: `https://data.geo.admin.ch/ch.meteoschweiz.ogd-smn/ogd-smn_meta_stations.csv`. An ER diagram can be seen in Figure 4.

Our goal was to determine the delays and cancellation rates of trains in both countries. To achieve this, we considered only data ranging from the beginning of September 2024 to the end of August 2025. Furthermore, we divided the data into two groups, one for each country, and analyzed delays per year, per month, per weather condition, and per canton or municipality. Additionally, we calculated the percentage of train cancellations per canton or municipality and identified which locations experience higher traffic on average. [6]

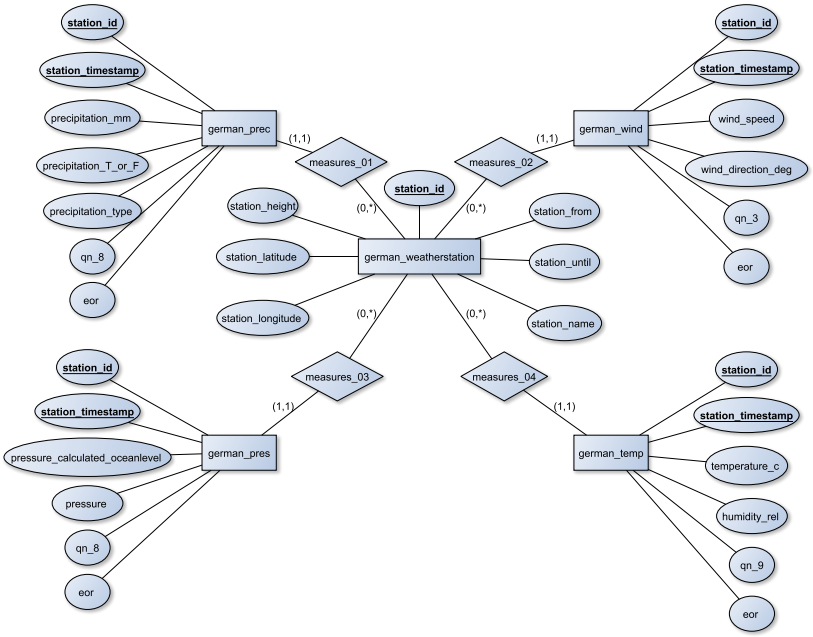


Fig. 2: ER Diagram of the german weather dataset

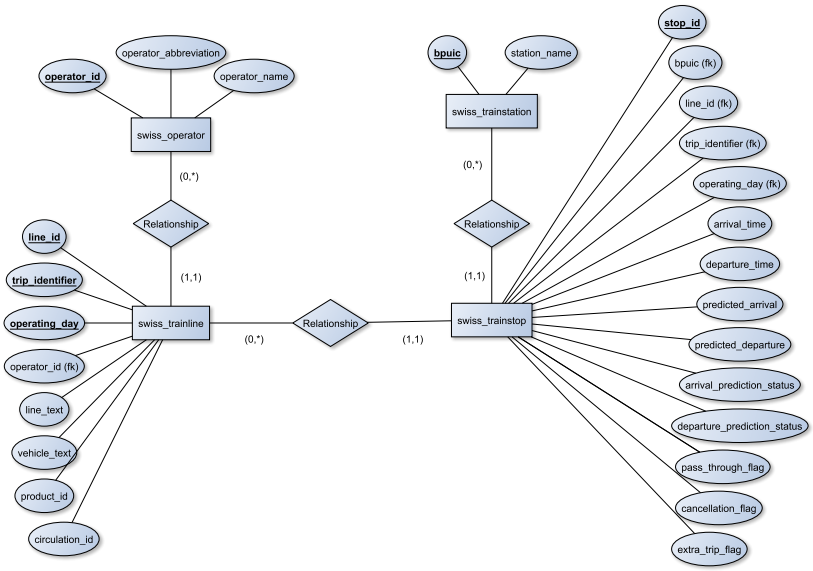


Fig. 3: ER Diagram of the swiss train dataset

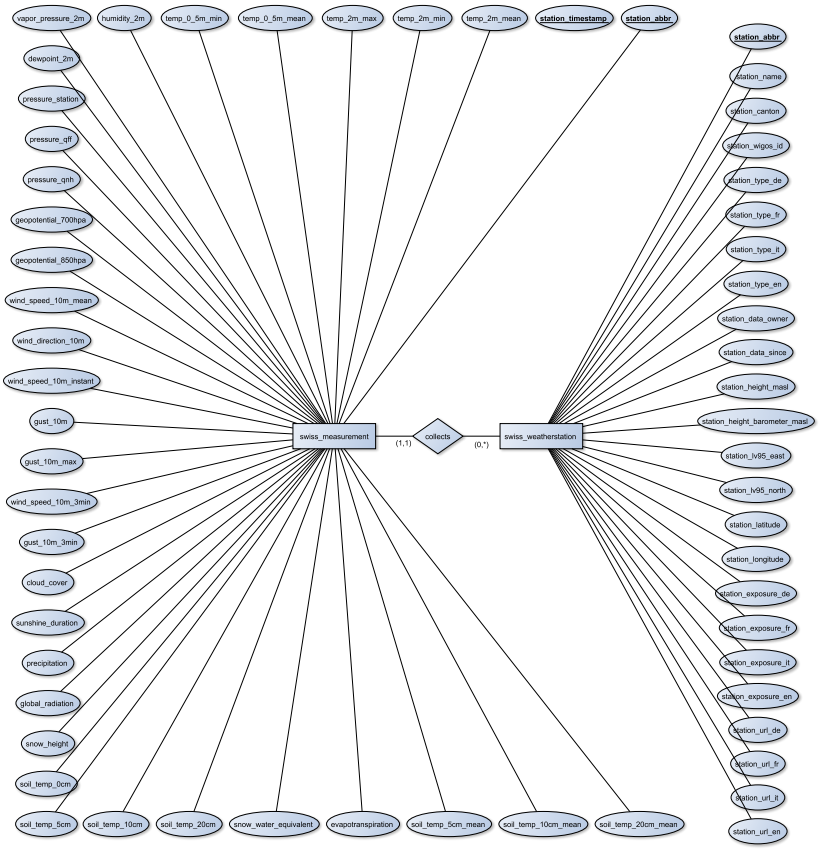


Fig. 4: ER Diagram of the swiss weather dataset

2 Integration

To integrate the data, we first dissected all four datasets and created corresponding ER diagrams. Following that, we attempted to create one large coherent schema which includes all data, but we ended up mostly leaving the datasets unconnected.

For the German train data, we had one `.parquet` file [1] for every month, which we turned into three tables. The `german_trainstation` table holds mostly just the station name but is important for classifying train events later. The `german_trainstop` table holds all information about train events, such as arrival and departure time, where the stop occurred, the delay in minutes, and information about cancellations. The `german_trainline` table holds information about the line under which the train operated, as well as the trains name, type, and its final destination.

The German weather data consisted of many files which were separated by type of measurement and whether the data was historic or recent. For each measurement, we created a table called `german_<measurement_name>` which holds the value of the measurement, additional information, the time the measurement was taken, and the weather station at which it was recorded. The `german_weatherstation` table holds information about the station name as well as its latitude and longitude.

For the Swiss train dataset, we had one set of large CSV files which we turned into four tables. The `swiss_trainstation` table holds mostly just the station name but is important for classifying train events later. The `swiss_trainstop` table holds information about all train stops that occurred, including arrival and departure time, location, and information about cancellations. The `swiss_trainline` table holds information about the train line which operated the train. It contains information about the train being used and the line name. It is important to note that we misinterpreted the dataset in this regard. Instead of just storing line information, we assumed lines differ from day to day, which resulted in a very convoluted primary key consisting of the line ID, a trip identifier, and the operating day. The `swiss_operator` table holds information about the company, such as SBB or ÖBB, under which a train completed its journey.

For the Swiss weather data, we had one set of CSV files which we implemented largely without modification as the table `swiss_measurement`. This table contains information about meteorological measurements, the date of the measurement, and which station recorded the measurement. Additionally, we created a table based on a CSV file of all Swiss weather stations called `swiss_weatherstations`. This table contains extensive data on the different weather stations, but we only use the latitude and longitude for each station.

The integrated schema, which holds all the data, contains all the aforementioned tables as well as one additional table called `trainstation_weatherstation`. This table holds information about which weather station is closest to each train station and also indicates whether the train station or weather station is located in Switzerland or Germany. This is useful for the final analysis. The full ER diagram of the integrated schema can be found in Figure 5, and the logical schema is listed below. Note that the table `trainstation_weatherstation` does not reference the actual tables and merely acts as a lookup table.

swiss_weatherstation(station_abbr, station_name, station_canton, station_wigos_id, station_type_de, station_type_fr, station_type_it, station_type_en, station_data_owner, station_data_since, station_height_masl, station_height_barometer_masl, station_lv95_east, station_lv95_north, station_latitude, station_longitude, station_exposure_de, station_exposure_fr, station_exposure_it, station_exposure_en, station_url_de, station_url_fr, station_url_it, station_url_en)

swiss_measurement(station_abbr, station_timestamp, temp_2m_mean, temp_2m_min, temp_2m_max, temp_0_5m_mean, temp_0_5m_min, humidity_2m, vapor_pressure_2m, dewpoint_2m, pressure_station, pressure_qff, pressure_qnh, geopotential_700hpa, geopotential_850hpa, wind_speed_10m_mean, wind_direction_10m, wind_speed_10m_instant, gust_10m, gust_10m_max, wind_speed_10m_3min, gust_10m_3min, cloud_cover, sunshine_duration, precipitation, global_radiation, snow_height, soil_temp_0cm, soil_temp_5cm, soil_temp_10cm, soil_temp_20cm, snow_water_equivalent, evapotranspiration, soil_temp_5cm_mean, soil_temp_10cm_mean, soil_temp_20cm_mean)

german_weatherstation(station_id, station_height, station_latitude, station_longitude, station_from, station_until, station_name)

german_temp(station_id, station_timestamp, qn_9, temperature_c, humidity_rel, eor)

german_prec(station_id, station_timestamp, qn_8, precipitation_mm, precipitation_T_or_F, precipitation_type, eor)

german_pres(station_id, station_timestamp, qn_8, pressure_calculated_oceanlevel, pressure, eor)

german_wind(station_id, station_timestamp, qn_3, wind_speed, wind_direction_deg, eor)

swiss_trainstation(bpuic, station_name)

swiss_operator(operator_id, operator_abbreviation, operator_name)

swiss_trainline(line_id, trip_identifier, operating_day, operator_id, line_text, vehicle_text, product_id, circulation_id)

swiss_trainstop(stop_id, bpuic, line_id, trip_identifier, operating_day, arrival_time, departure_time, predicted_arrival, predicted_departure, arrival_prediction_status, departure_prediction_status, pass_through_flag, cancellation_flag, extra_trip_flag)

german_trainstation(eva, station_name, xml_station_name)

german_trainline(train_line_ride_id, final_destination_station, train_name, train_type)

german_trainstop(stop_id, eva, train_line_ride_id, final_destination_station, train_name, arrival_change_time, departure_change_time, arrival_planned_time, departure_planned_time, delay_in_min, train_time, is_canceled, train_line_station_num)

trainstation_weatherstation(trainstation, trainstation_in_ch, weatherstation, weatherstation_in_ch)

To integrate the schema, we used the same approach for each dataset. This

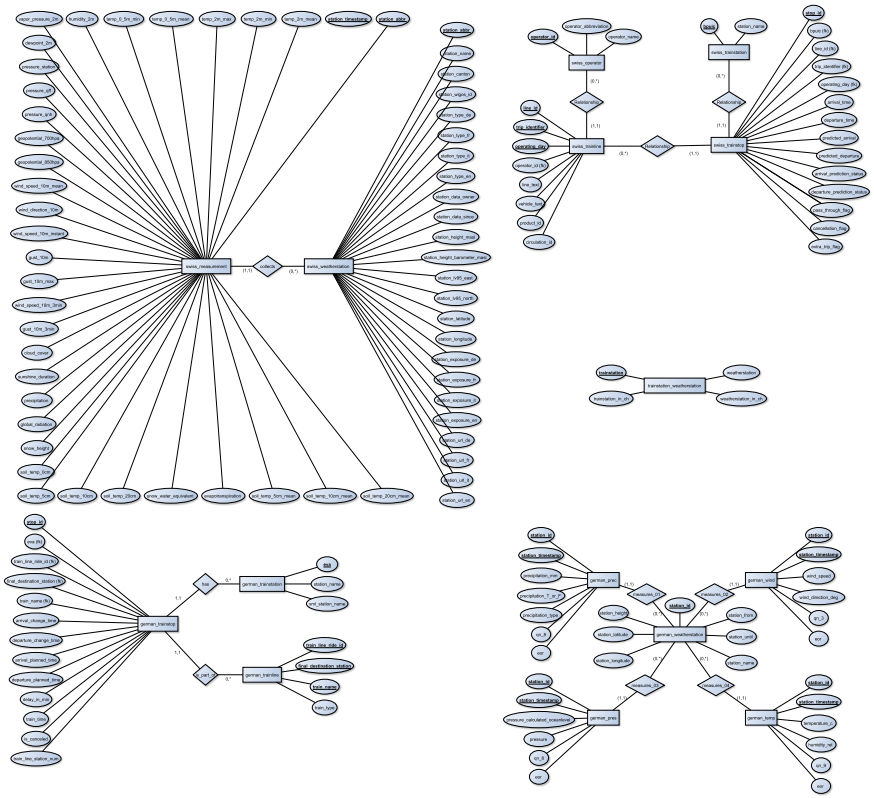


Fig. 5: ER Diagram of the final database

included loading one source file into a pandas DataFrame, loading any CSV files which needed to be extended (e.g., the `swiss_trainstation` CSV), discarding all rows which were missing a necessary column value such as the primary key, creating DataFrames for each table related to that CSV, filling the DataFrames with data from the source file or extending them using concatenation and deduplication, performing a final deduplication step to catch any remaining duplicates, and storing all necessary DataFrames either in a new CSV file or in the CSV from which the data was loaded. After one file was processed, we repeated this procedure until all files had been iterated over. All resulting CSV files could then easily be loaded into the database using the `COPY` command. [9] This was significantly faster than performing inserts for each individual data point and

took at most 10 to 15 minutes. The processes differed slightly for each dataset, which we discuss in the following sections.

For the German train data, we first needed to convert the monthly source files from Parquet to CSV to avoid pandas DataFrames containing columns with the `object` data type.

For the German weather data, we had a large number of files which were separated into folders by temperature, precipitation, pressure, and wind speed, as well as by historical data ranging up to 2025 and recent data ranging from the beginning of 2025 until now. Each folder contained one subfolder per weather station, holding both the measurement data and the station data. This significantly complicated the process, as we had to iterate over all possible file locations to locate all relevant data. Furthermore, all data was stored in text files formatted similarly to CSV files but containing additional whitespace. These whitespaces had to be removed before processing the data. Additionally, we needed to filter the data to the correct time span, as data from outside our specified period was present. To clean and normalize the data, cast some columns from float to int or from int to boolean to correctly represent the measurements, set impossible data points to NA, and format dates to match the PostgreSQL date and timestamp format. For the station data, we additionally retained only the most recently recorded metadata to avoid duplicated weather stations.

For the Swiss train data, we used an artificial ID in the form of an increasing integer. This required us to load and store the current ID value in a JSON file for each CSV processed. Additionally, we filtered out all non-train events, as they did not fit our specified analysis goals and would have significantly increased the dataset size. Originally, the raw SBB data amounted to approximately 170 GB. We also formatted dates to ensure compatibility with the PostgreSQL date and timestamp format.

For the Swiss weather data, we similarly formatted dates to the PostgreSQL date and timestamp format and filtered the data to the correct time span, as the dataset also contained measurements outside our specified period.

The `trainstation_weatherstation` table was populated only after all other tables had already been filled. We loaded all train stations and weather stations into pandas DataFrames to determine which weather station was closest to each train station. Fortunately, the weather stations in both the German and Swiss datasets were documented with latitude and longitude. Therefore, we only needed to determine the coordinates of the train stations. To achieve this, we used the Nominatim interface [3] of `geopy` to locate the train stations. We discarded all stations that were not located in Germany or Switzerland, as our analysis focused exclusively on these countries. This approach worked well in most cases; however, for approximately 40 train stations, the locations had to

be determined manually. Afterwards, we constructed a k-d tree [8] using `scipy`, a data structure designed to organize points in k dimensions, which in our case was two. Using built-in functions, we then identified the closest weather station for each train station and stored the resulting data in a CSV file, which was again inserted into the database using the `COPY` command. [6]

3 API

For the API, FastAPI was used to implement a REST interface for accessing the precipitation data stored in the german_prec table. The API provides basic reading (GET), adding (POST), modifying (PUT) and deleting (DELETE) functionality. The service connects directly to the database using psycopg2.
The API can be started locally using:

```
python -m uvicorn main:app --reload
```

Alternatively, it can be started via the provided `run_fastapi.bat` file.

After startup, the interactive API documentation is available at:

```
http://127.0.0.1:8000/docs
```

3.1 Data Model

All requests that insert or update data use the same JSON structure, defined using a Pydantic model. Each record is uniquely identified by a composite primary key consisting of the station_id and the station_timestamp. All remaining attributes are optional.

Example JSON object:

```
{
  "station_id": 164,
  "station_timestamp": "2024-09-01T00:00:00",
  "qn_8": 3,
  "precipitation_mm": 0,
  "precipitation_t_or_f": false,
  "precipitation_type": 0,
  "eor": "eor"
}
```

3.2 GET: Read Precipitation Data

Endpoint:

```
GET /precipitation/{station_id}
```

This endpoint returns up to 100 precipitation records for a given weather station. The limit is applied to prevent excessive data transfer and ensure acceptable response times.

Example request:

```
GET /precipitation/164
```

Example response:


```
[
  {
    "station_id": 164,
    "station_timestamp": "2024-09-01T00:00:00",
    "qn_8": 3,
    "precipitation_mm": 0,
    "precipitation_t_or_f": false,
    "precipitation_type": 0,
    "eor": "eor"
  }
]
```


3.3 POST: Insert New Data

Endpoint:

```
POST /precipitation
```

This endpoint inserts a new precipitation record into the database.

Example request body:


```
{
  "station_id": 1,
  "station_timestamp": "2024-09-01T00:00:00",
  "qn_8": 3,
  "precipitation_mm": 0,
  "precipitation_t_or_f": false,
  "precipitation_type": 0,
  "eor": "eor"
}
```


Example response:


```
{
  "message": "new tuple created"
}
```

3.4 PUT: Update Existing Data

Endpoint:

`PUT /precipitation/{station_id}/{station_timestamp}`

This endpoint updates an existing precipitation record identified by station ID and timestamp. The primary key itself cannot be modified.

Example request:

`PUT /precipitation/1/2024-09-01T00:00:00`

Example request body:

```
{
  "station_id": 1,
  "station_timestamp": "2024-09-01T00:00:00",
  "qn_8": 3,
  "precipitation_mm": 5,
  "precipitation_t_or_f": true,
  "precipitation_type": 1,
  "eor": "eor"
}
```

Example response:

```
{
  "message": "updated"
}
```

3.5 DELETE: Remove Data

Endpoint:

`DELETE /precipitation/{station_id}/{station_timestamp}`

This endpoint deletes a precipitation record using station ID and timestamp as identifiers. In the following example, the artificially created entry from the POST example is removed.

Example request:

`DELETE /precipitation/1/2024-09-01T00:00:00`

Example response:

```
{
  "message": "deleted"
}
```

4 Analysis

References

[1] *Apache Parquet*. https://parquet.apache.org/. 2025.
[2] *Ist-Daten Archiv*. https://archive.opentransportdata.swiss/actual_data_archive.htm. 2025.
[3] *Nominatim — OpenStreetMap Search Engine*. https://nominatim.org/. 2025.
[4] *Open Data Download Documentation*. https://opendatadocs.meteoswiss.ch/de/general/download. 2025.
[5] *Open Data Portal*. https://opendata.dwd.de/. 2025.
[6] OpenAI. *ChatGPT (GPT-5) language model*. https://chat.openai.com/. Used to assist with spelling, grammar, and rephrasing of LaTeX excerpts in this project. 2026.
[7] piebro. *Deutsche Bahn Data*. https://github.com/piebro/deutsche-bahn-data. 2025.
[8] *scipy.spatial.KDTree — SciPy v1 Reference Guide*. SciPy Community. 2025. URL: https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.KDTree.html.
[9] The PostgreSQL Global Development Group. *SQL COPY — PostgreSQL Documentation*. PostgreSQL Global Development Group. 2025.